

GeoTaichi AdaptiveGrid 单次位移罚约束实现说明

GeoTaichi implementation note

2026 年 6 月 16 日

1 实现目标

GeoTaichi 的 AdaptiveGrid 是两层 2:1 自适应背景网格：大部分区域使用粗网格，只在满足准则的局部区域使用细网格。界面处需要控制粗细网格之间的速度/位移不连续，否则会出现悬点振荡或粗细层传递误差。

本实现的目标是：

- 默认只做一次界面约束，不使用多次迭代；
- 约束参数由材料刚度、粒子体积、网格尺寸、时间步和节点有效质量标定；
- 不增加全局线性求解、不增加持久的界面材料点副本，保持 AdaptiveGrid 的速度优势；
- 保留旧式经验松弛模式，便于复现实验和回退。

2 为什么不做硬位置投影

显式 MPM 中，背景网格是每个时间步重置的计算网格，不是随材料运动的实体网格。真正持久的位置自由度是粒子位置 $\mathbf{x}_p$ ，而悬点只是当前时间步内由粗细网格关系生成的约束节点。

直接做位置投影有两个选择，但都不适合作为当前默认实现：

- 投影背景节点位置：背景节点下一个时间步会重置，节点位置本身不是状态量。这样做不能形成稳定的历史位移约束。
- 投影粒子位置：会立即改变粒子形函数支持、体积更新、速度梯度、应力更新和动量一致性。要严谨实现，需要同步重建形函数、修正应变/体积历史，并处理能量和动量一致性，工程代价较高。

因此，本实现采用单次位移罚约束：不直接硬改粒子坐标，而是把本步粗细层速度差解释为一步内的预测位移差，

$$\mathbf{D} \approx \Delta t \mathbf{g}_v,$$

再由位移罚能得到节点速度修正。这样保留了位置罚的尺度含义，同时仍是局部一次冲量更新。

3 约束变量

代码中约束作用在已经由质量归一化后的节点速度上。字段名仍为 `node.momentum`，但在约束阶段它表示节点速度。

3.1 BridgingDomain

对跨粗细层支持域的粒子，定义粗层和细层插值得到的速度差：

$$\mathbf{g}_v = \mathbf{v}_c - \mathbf{v}_f.$$

其中

$$\mathbf{v}_c = \sum_{i \in c} \tilde{N}_i \mathbf{v}_i, \quad \mathbf{v}_f = \sum_{j \in f} \tilde{N}_j \mathbf{v}_j.$$

$\tilde{N}$ 是按 bridging 权重归一化后的形函数权重。

3.2 悬点约束

对悬点 s ，定义悬点速度和 master 节点插值速度的差：

$$\mathbf{g}_v = \mathbf{v}_s - \sum_m w_m \mathbf{v}_m.$$

两类约束统一写成

$$\mathbf{g}_v = J\mathbf{v},$$

对应的 inverse effective mass 为

$$W = JM^{-1}J^T = \sum_i \frac{J_i^2}{m_i}.$$

4 单次位移罚公式

局部罚刚度取为

$$k = \beta \frac{E\sqrt{V}}{h}.$$

其中 E 是材料 Young 模量， V 是参考体积， h 是细网格最小尺寸， β 是无量纲放大系数。

用一步预测位移差 $\mathbf{D} \approx \Delta t \mathbf{g}_v$ 写罚能：

$$\Pi_p = \frac{1}{2} k \|\mathbf{D}\|^2.$$

在当前显式速度投影框架中，采用有界的单次松弛系数：

$$\gamma = \frac{k\Delta t^2 W}{1 + k\Delta t^2 W}, \quad 0 \leq \gamma < 1.$$

最终冲量为

$$\mathbf{j} = \gamma \frac{\mathbf{g}_v}{W}.$$

该形式有三个性质：

- 当材料更硬、时间步更大或有效质量更小时， γ 自动增大；
- γ 有上界，不会因为罚刚度过大直接变成不稳定的硬投影；
- 一次计算即可得到本步修正量，不需要重复迭代降低残差。

代码中还使用 `Penalty` 作为 γ 的上限：

$$\gamma \leftarrow \min(\gamma, \text{Penalty}).$$

默认 `Penalty=1.0`，即不额外削弱 scaled penalty。

5 参考体积和材料模量

`BridgingDomain` 有明确的跨层粒子，因此使用粒子体积：

$$V = V_p.$$

悬点约束没有独立的界面粒子，因此使用细网格单元体积：

$$V = \prod_d h_f^{(d)}.$$

默认情况下，元素会从材料表中读取最大的 `young` 作为代表性 E 。如果需要手动指定，可在 `AdaptiveGrid` 中设置 `PenaltyYoungModulus`。当材料表没有可用的 `Young` 模量且用户也未指定时，scaled 模式会退化为 `Penalty` 控制的旧式松弛。

6 代码实现位置

文件	作用
<code>AdaptiveQuadrilateralKernel.py</code>	2D scaled penalty 和约束冲量组装
<code>AdaptiveHexahedronKernel.py</code>	3D scaled penalty 和约束冲量组装
<code>AdaptiveQuadrilateralElement.py</code>	2D 参数解析、材料模量配置、kernel 调用
<code>AdaptiveHexahedronElement.py</code>	3D 参数解析、材料模量配置、kernel 调用
<code>ULExplicitEngine.py</code>	在显式时间步中调用 <code>AdaptiveGrid</code> 约束

2D 和 3D kernel 的核心函数分别为:

scaled_penalty_relaxation_2d, scaled_penalty_relaxation.

它们计算 k 、 $k\Delta t^2 W$ 和 γ ，随后由以下 kernel 生成节点冲量:

- assemble_bridging_penalty_impulse
- assemble_hanging_penalty_impulse_list

在 apply_hanging_penalty_impulse 中，节点速度修正为

$$\Delta \mathbf{v}_i = \frac{\mathbf{j}_i}{m_i},$$

并同步修正节点加速度:

$$\Delta \mathbf{a}_i = \frac{\Delta \mathbf{v}_i}{\Delta t}.$$

这样 G2P 的 PIC/FLIP 速度更新都会看到同一个约束修正。

7 效率优势和适用条件

AdaptiveGrid 的速度优势来自局部精化，而不是来自约束本身。与全域 fine uniform grid 相比，自适应网格只在局部区域增加细节点和细粒子。计算量主要由以下部分组成:

$$O(N_p n_s) + O(N_g) + O(N_\Gamma),$$

其中 N_p 是粒子数， n_s 是单粒子支持节点数， N_g 是活动网格节点数， N_Γ 是粗细界面约束数。

当前实现为了保持速度优势做了三点限制:

- 默认 PenaltyIterations=1，约束 kernel 只执行一次;
- RefineRatio 限制最大精化单元比例，避免退化为全域 fine grid;
- 只增加少量标量计算，不增加全局矩阵、局部线性求解或额外持久粒子副本。

实现中还做了两项直接影响效率的处理。首先，网格重构按 refinement 周期触发，粒子分裂扫描按 refinement/split 周期或粒子数变化触发，避免每个时间步都扫描全部粒子。其次，在 2D 问题尚未出现 refined cell 且未启用 BridgingDomain 时，形函数映射直接走粗网格路径，不再查询细网格逻辑映射表。这使启用 AdaptiveGrid 但暂未触发精化的算例接近 uniform coarse grid 的成本。

因此，当精化区域明显小于全域、且 RefineInterval 不过小的时候，AdaptiveGrid 应当比等效 fine uniform grid 更快。若 RefineRatio 接近 1，或每步都大范围重构精化区，自适应开销可能抵消优势。推荐把 RefineRatio 控制在问题允许的较小范围内，例如 0.2 ~ 0.4，并用 smoke/benchmark 记录 P2G、Grid kinematic、G2P 和 Contact & Kinematic boundary 时间。

8 参数建议

参数	默认值	建议
PenaltyMode	Scaled	默认使用单次位移罚标定
PenaltyBeta	10.0	一般从 10 开始，界面跳跃偏大时增大
Penalty	1.0	作为 γ 上限，通常保持 1.0
PenaltyIterations	1	默认单次；只有诊断显示必要时才增加
PenaltyYoungModulus	0.0	默认自动读取材料 Young 模量
ParticleSplitInterval	RefineInterval	控制粒子分裂扫描频率

推荐配置：

```
PenaltyMode = "Scaled", PenaltyBeta = 10, PenaltyIterations = 1.
```

若粒子会频繁穿过 refined 区域，可把 ParticleSplitInterval 设小；若主要关注速度，保持默认值，让粒子分裂扫描和网格 refinement 同步。

如需复现旧算法：

```
PenaltyMode = "Relaxation", Penalty = 0.8, PenaltyIterations = 2.
```

9 结论

当前实现没有采用硬位置投影，因为硬投影需要引入持久位移状态或直接移动粒子，会显著增加复杂度并影响显式 MPM 的一致性。现有方案采用单次位移罚约束：

$$\mathbf{D} \approx \Delta t \mathbf{g}_v, \quad k = \beta \frac{E\sqrt{V}}{h}, \quad \gamma = \frac{k\Delta t^2 W}{1 + k\Delta t^2 W}.$$

它保留了位置罚的尺度含义，同时仍是局部一次冲量更新。对当前 AdaptiveGrid 的主要目标而言，这比硬位置投影更适合：实现简单、参数可解释、约束次数少，并能保持相对于 fine uniform grid 的局部精化速度优势。